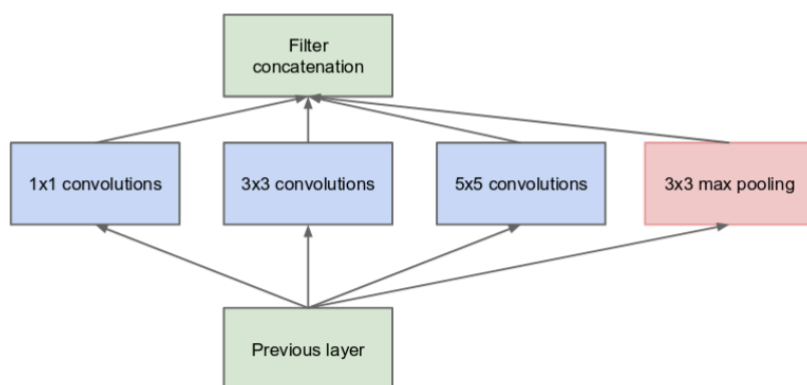


- 由于信息位置的这种巨大变化，为卷积操作选择**正确的内核大小**变得困难。**大内核有益于捕获全局信息，而更小的内核有利于获取局部信息。**
- **非常深的网络容易过度拟合。**它也很难通过整个网络传递梯度更新。
- 简单地堆叠大卷积运算在**计算上是昂贵的**。

解决方案：

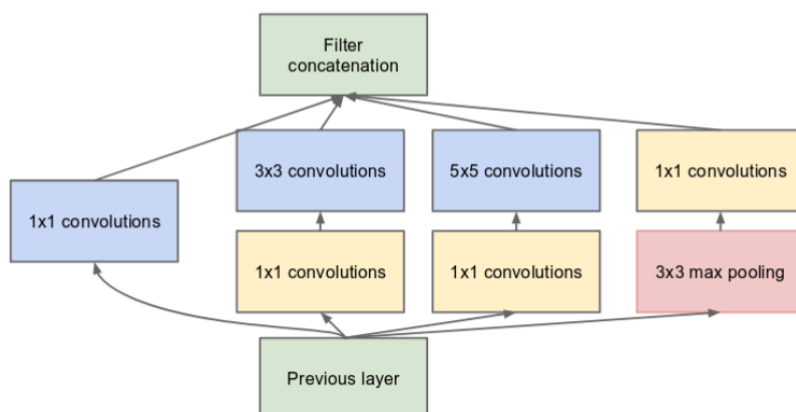
为什么不在**同一级别上运行多种尺寸的过滤器**？网络本质上会变得更“**宽**”而不是“**更深**”。作者设计了Inception模块以反映相同的内容。

下图是“简单”的Inception模块。它在输入上执行具有**3种不同大小的滤波器**（1x1,3x3,5x5）的卷积。此外，还执行**最大池化**。它们的输出被**连接**并发送到下一个Inception模块。



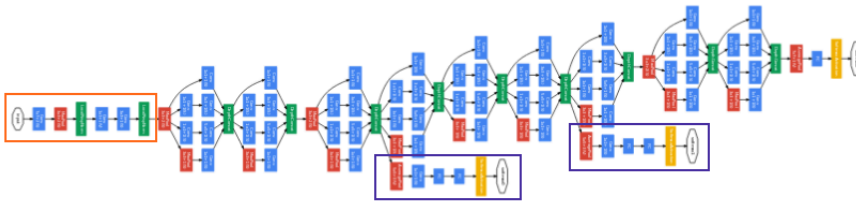
(a) Inception module, naïve version

如前所述，深度神经网络在**计算上是昂贵的**。为了降低成本，作者通过在3x3和5x5卷积之前添加**额外的1x1卷积来限制输入通道的数量**。虽然增加一个额外的操作似乎会导致更多的计算，但事实上1x1卷积比5x5卷积便宜得多，输入通道数量减少也使计算减少。但请注意，1x1卷积是在最大池化层之后引入的，而不是之前引入的。



(b) Inception module with dimension reductions

使用降维Inception模块构建神经网络架构。这通常被称为GoogLeNet（Inception v1）。架构如下所示：



GoogLeNet。橙色的框是主干（stem），它包含一些基础的卷积。紫色框是辅助分类器。宽的部分是Inception模块。（来源：Inception v1）

GoogLeNet有9个这样的Inception模块线性堆叠。它是22层深（27层，包括池化层）。它在最后一个Inception模块的末尾使用全局平均池。

可想而知，它是一个非常深的分类器。与任何非常深的网络一样，它受到梯度消失问题的影响。

为了防止网络的中间部分“消失”，作者引入了两个辅助分类器（图像中的紫色框）。它们基本上将softmax应用于两个Inception模块的输出，并计算相同标签上的辅助损失（auxiliary loss）。总损耗函数是所述的辅助损失和实际损失（real loss）的加权和。对于每种辅助损失，论文中使用的权重值为0.3。

```
# 训练时InceptionNet使用的总损失。  
total_loss = real_loss + 0.3 * aux_loss_1 + 0.3 * aux_loss_2
```

毋庸置疑，辅助损失纯粹用于训练目的，在推理过程中被忽略。

InceptionNet v2

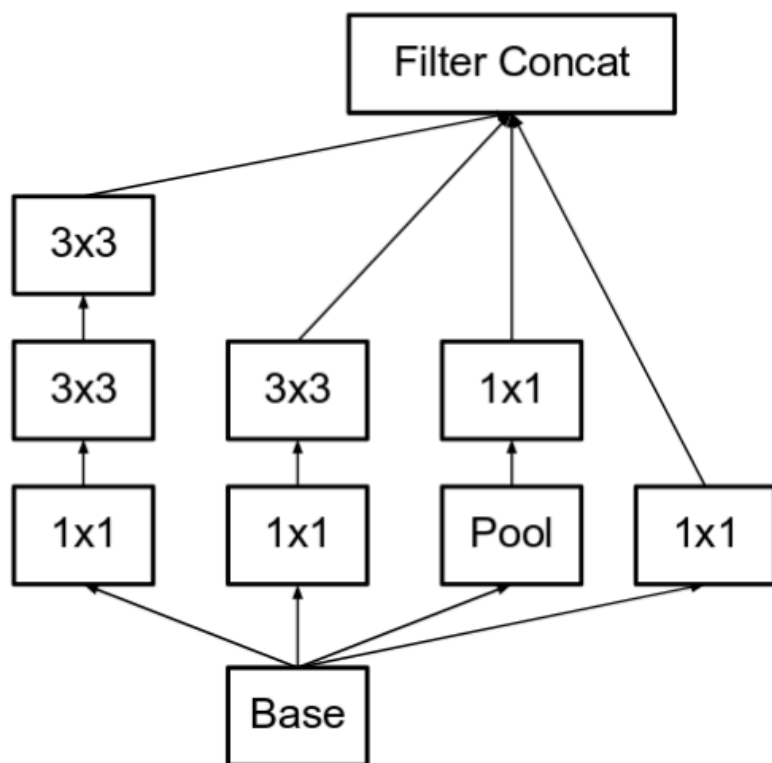
InceptionNet v2和InceptionNetv3在同一篇论文中提出。作者提出了许多升级，这些升级提高了准确性并降低了计算复杂度。InceptionNet v2探讨了以下内容：

前提：

- 减少代表性瓶颈。大概思想是，当卷积没有大幅度降低输入的维度时，神经网络表现会更好。过度减小尺寸可能会导致信息丢失，称为“代表性瓶颈（representational bottleneck）”
- 使用智能分解方法，可以在计算复杂性方面使卷积更有效率。

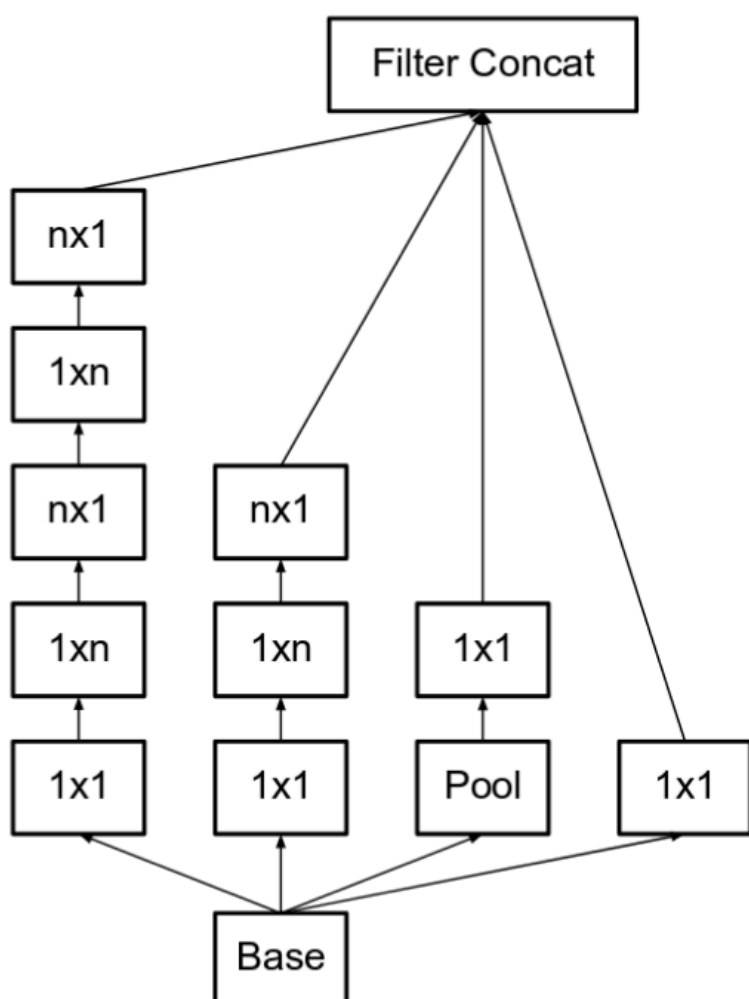
解决方案：

- 将5x5卷积分解为两个3x3卷积运算，以提高计算速度。虽然这看似违反直觉，但5x5卷积比3x3卷积贵2.78倍。因此，堆叠两个3x3卷积实际上可以提高性能。这在下图中说明。



旧的Inception模块的最左侧5x5卷积现在表示为两个3x3卷积。（来源：Inception v2）

此外，它们将滤波器大小 $n \times n$ 的卷积分解为 $1 \times n$ 和 $n \times 1$ 卷积的组合。例如，3x3卷积相当于首先执行 1×3 卷积，然后在其输出上执行 3×1 卷积。他们发现这种方法比单个3x3卷积便宜33%。这在下图中说明。



这里 $n=3$ ，这个图与上个图有着同等的效果。最左边的 5×5 卷积可以表示为两个 3×3 卷积，而 3×3 卷积可以表示为 1×3 和 3×1 串联。（来源：Inception v2）

- 模块中的**滤波器组被扩展**（更宽而不是更深）以消除代表性瓶颈。如果模块变得更深，则尺寸会过度减少，从而导致信息丢失。这在下图中说明
- 上述三个原则用于构建三种不同类型的Inception模块（让我们按照它们的介绍顺序把它们称为模块**A**，**B**和**C**。这些名称是为了清晰起见而不是正式名称）。架构如下：

type	patch size/stride or remarks	input size
conv	$3\times 3/2$	$299\times 299\times 3$
conv	$3\times 3/1$	$149\times 149\times 32$
conv padded	$3\times 3/1$	$147\times 147\times 32$
pool	$3\times 3/2$	$147\times 147\times 64$
conv	$3\times 3/1$	$73\times 73\times 64$
conv	$3\times 3/2$	$71\times 71\times 80$
conv	$3\times 3/1$	$35\times 35\times 192$
$3\times$ Inception	As in figure 5	$35\times 35\times 288$
$5\times$ Inception	As in figure 6	$17\times 17\times 768$
$2\times$ Inception	As in figure 7	$8\times 8\times 1280$
pool	8×8	$8\times 8\times 2048$
linear	logits	$1\times 1\times 2048$
softmax	classifier	$1\times 1\times 1000$

这里，“figure 5”是模块A，“figure 6”是模块B，“figure 7”是模块C。（来源：Inception v2）

Inceptionv3

前提

- 作者指出，**辅助分类器**在训练过程即将结束时准确度接近饱和时才会有很大贡献。他们认为它们可以作为**正规化**，特别是如果它们具有BatchNorm或Dropout操作。
- 研究在不大幅度改变模块的情况下改进Inception v2的可行性。

解决方案

- Inception Net v3**包含了为Inception v2规定的所有上述改进，另外还使用了以下内容：

1. **RMSProp优化器⁺**。
2. 分解 7×7 卷积。
3. 辅助分类器中的BatchNorm。
4. 标签平滑（添加到损失公式中的一种正规化组件，可防止网络对类过于自信。防止过度拟合）。

Inception v4

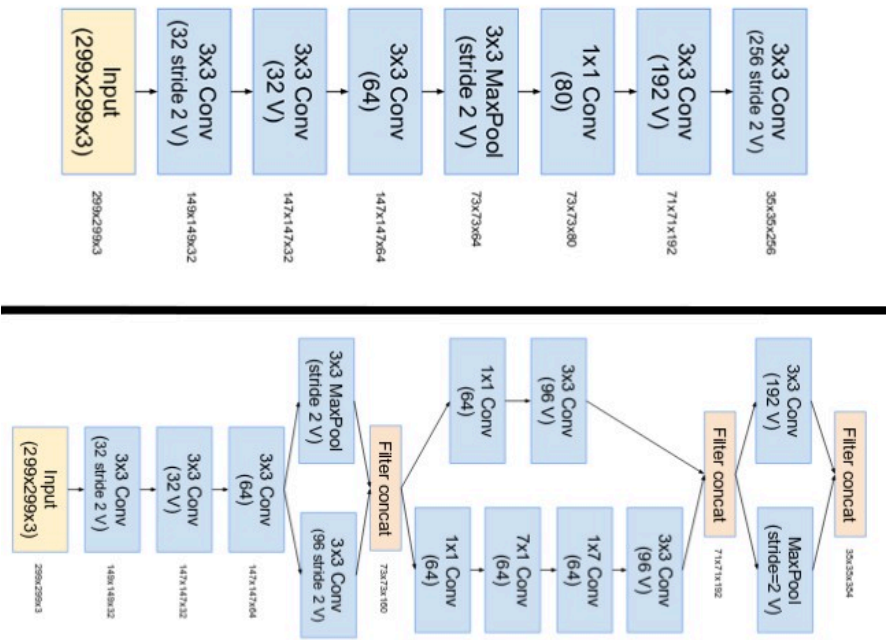
在同一篇论文中介绍了Inception 版本v4和Inception-ResNet。为清楚起见，让我们在不同的章节中讨论它们。

前提

- 使模块更统一。作者还注意到一些模块没必要这么复杂。这可以使我们通过添加更多这些统一模块来提高性能。

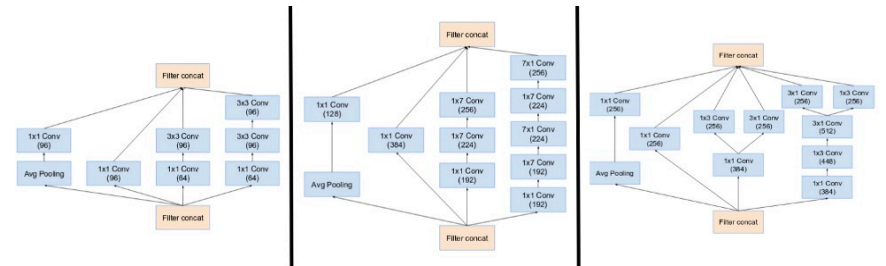
解决方案

- Inception v4的“主干（stem）”被修改了。这里的stem是指在引入Inception块之前执行的初始操作集。



顶部图像是Inception-ResNet v1的主干。底部图像是Inception v4和Inception-ResNet v2的主干。
(来源：Inception v4)

- 他们有三个主要的Inception模块，名为A，B和C（与Inception v2不同，这些模块实际上名为A，B和C）。它们看起来与它们的Inception v2（或v3）对应部分非常相似。



（左起）Inception v4中使用的Inception模块A，B，C。请注意它们与Inception v2（或v3）模块的相似程度。（来源：Inception v4）

- Inception v4引入了专门的“Reduction Blocks”，用于改变网格的宽度和高度。早期版本没有显式Reduction Blocks，但实现了类似功能。

Inception-ResNet v1和v2

受ResNet性能启发，提出了一种混合Inception模块。Inception ResNet有两个子版本，即v1和v2。在我们查看显著特征之前，让我们看一下这两个子版本之间的细微差别。

- Inception-ResNet v1的计算成本与Inception v3类似。
- Inception-ResNet v2的计算成本与Inception v4类似。

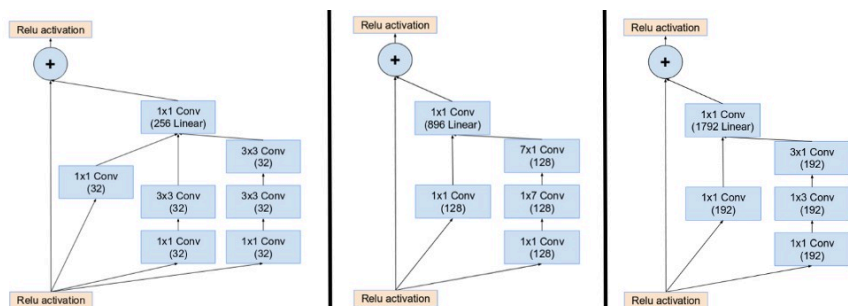
- 它们有**不同的主干**，如Inception v4部分所示。
- 两个子版本对于**模块A, B, C和Reduction Blocks**具有**相同的结构**。唯一的**区别是超参数**设置。在本节中，我们将只关注结构。有关确切的超参数设置，请参阅该文章（图像为Inception-Resnet v1）。

前提

- 引入将Inception模块的卷积运算的输出添加到输入的残差连接。

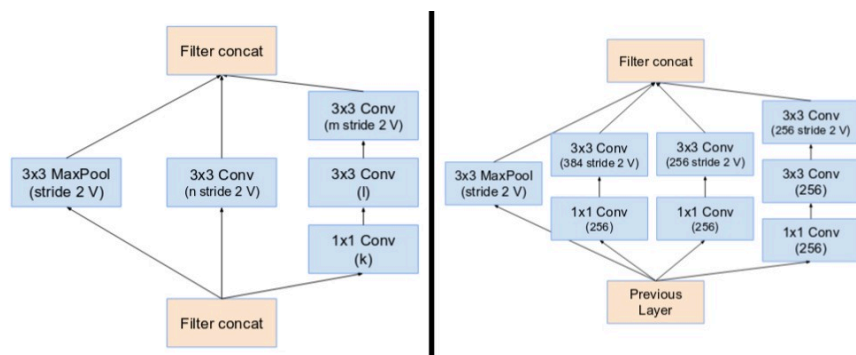
解决方案

- 为了实现残差加法，卷积后的输入和输出必须具有相同的尺寸。因此，我们在Inception卷积之后使用1x1卷积来匹配深度大小（卷积后的深度增加）。



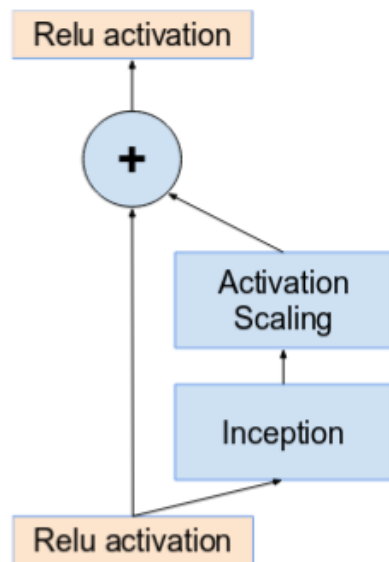
左起) Inception ResNet中的Inception模块A, B, C。注意池化层如何被残差连接替换，以及在添加之前还有额外的1x1卷积。（来源：Inception v4）

- 主要Inception模块内的池化操作被替换为有利于残差连接。但是，您仍然可以在reduction blocks中找到这些操作。Reduction Block A与Inception v4中的相同。



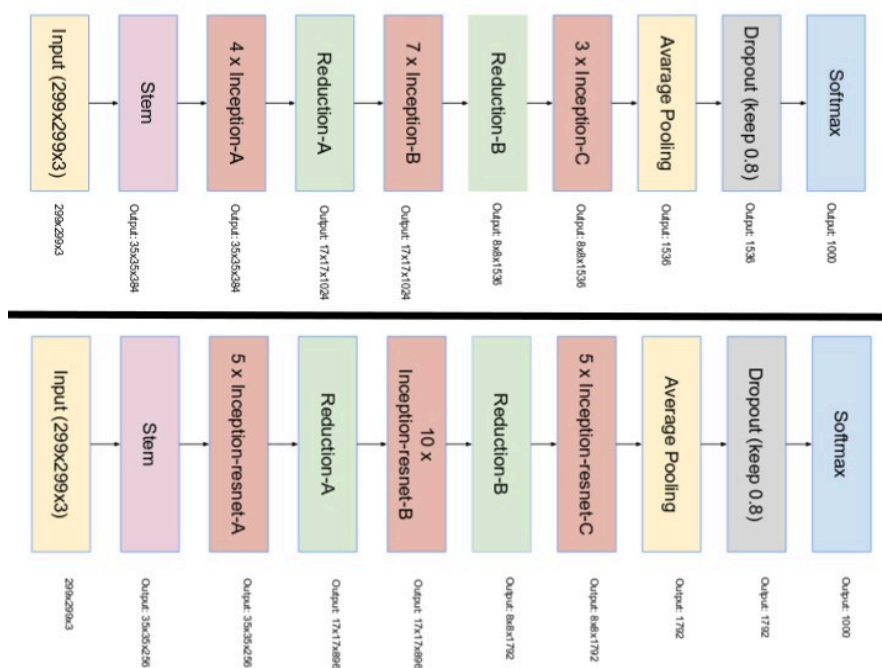
（从左侧）Reduction block A（尺寸35x35减小至17x17）和Reduction Block B（尺寸17x17减小至8x8）。有关精确的超参数设置（V, l, k），请参阅论文。（来源：Inception v4）

- 如果滤波器的数量超过1000，那么网络架构中更深的残差单元会导致网络“死亡”。因此，为了增加稳定性，作者将残差激活量调整，系数为0.1到0.3。



激活按常量缩放以防止网络死亡。(来源: Inception v4)

- 为了在单个GPU上训练模型，原始论文在求和之后没有使用BatchNorm（以使整个模型适合单个GPU）。
- 结果发现，Inception-ResNet模型能够再用较少的epoch时获得更高的精度。
- Inception v4和Inception-ResNet的最终网络布局如下：



本文翻译自: towardsdatascience.com/...

发布于 2018-09-24 10:12

深度学习 (Deep Learning) 神经网络

3 条评论

默认 最新



ayo

jianshu.com/p/6d66fa4ca... 可以对比看

2023-11-22

回复 喜欢



张贤同学

什么垃圾

2020-06-13

回复 喜欢



苏物生

贵/便宜 的翻译看着有点难受，简单修改一下也行呀，纯机翻的吧

2020-06-09

回复 喜欢